
Practical 1: Festival Road Closings

ALGORITHMS AND DATA STRUCTURES

PRACTICAL 1

Group number: 42

Lan Stare

s1169977

1 Explanation of the Algorithm

1.1 Initialization

Reading in the data

We read in the data by first saving the entire input as a list of integers using the `sys` library. This allows us to easily split the flattened row into integers regardless of the format of the `.in` files, since some files only have the number of vertices ($|V|$) and edges ($|E|$) in the first line (and every subsequent line contains exactly one edge) while others have all the data in a single line. After reading in the data, we extract $|V|$ and $|E|$ from the list of integers we just created. Finally, we build a list of edge tuples by iterating through the list of integers and grouping every three integers into a tuple representing an edge $(v1, v2, type)$.

Initializing data structures

The main data structure we use to store our graph is a list of tuples, where each tuple represents an edge in the graph. Each tuple contains three elements: the starting vertex, the ending vertex, and the type of edge we have. Type 0 represents a pedestrian-only road, type 1 represents a bus-only road and type 2 represents a road that can be used by both pedestrians and busses (later referred to as a combined road). We use this data structure because it allows us to easily iterate through all the edges and we also don't need to keep track of adjacent vertices, which would be possible with an adjacency list or matrix but would be more expensive. Duplicate edges are also possible in the input, so dictionaries are not suitable for this problem.

1.2 Algorithm Logic

Algorithm: Initialization

The core idea is using Kruskal's algorithm. Since we haven't separated the graph into a pedestrian and bus graph prior to running the algorithm, we have to take care of both (or all three) types of edges while iterating.

We first build two union-find data structures, one for the pedestrian graph and one for the bus graph. This allows us to keep track of connected components for both types. We then need to build a find function that finds the root of a vertex in the union-find data structure and a union function that connects two vertices in the union-find if they are not already in the same component (their roots are different).

Before iterating through the edges, we need to sort them by their type. We want to prioritize the combined edges (type 2), so we assign them weight 1 while assigning weight 2 to the pedestrian-only and the bus-only edges (types 0 and 1).

Finally, we initialize a variable `kept_edges` to 0, which will keep track of the number of edges we keep in the final graph.

Algorithm: Main loop

Up until now, we have only implemented the initiation of the Kruskal's algorithm. The main loop is where we execute our modifications to the original algorithm.

First we add a switch set to False that will keep track of whether we have added an edge in the current iteration or not. We then iterate through the sorted list of edges. If the road is of type 1 or 2 (bus-only or combined), we check if the two vertices of the edge are in different components in the bus union-find data structure. If they are, we connect them and set the switch to True. We do the same if the road is of type 0 or 2 (pedestrian-only or combined). If the switch is set to True after checking both if statements, we increment the *kept_edges* by 1.

Arguments for used structures and algorithms

We use Kruskal's algorithm because we want to find the minimum amount of edges that connect all vertices in both graphs. The graph is undirected so we could have used Prim's algorithm as well with binary heaps (to get the same time complexity) but Kruskal's algorithm is easier to implement and modify for our needs.

Instead of only checking whether adding an edge would create a cycle in one graph, we have to check both graphs. To do this efficiently, we separated the union-find data structures for both graphs, which allows us to keep track of connected components for both graphs at the same time.

Later we also added an extra stopping criterion to the main loop that checks whether both graphs are fully connected, further explained in the next section.

Algorithm 1 Pseudocode example.

```
1: function FUNCTION(arguments)
2:   Step 1
3:   while Condition 1 does not hold do F
4:     if Condition 2 then
5:       return Yes
6:     else
7:       Step 2
8:   return No
```

1.3 Returning the Answer

The original algorithm terminates when we have iterated through all the edges. However, we added a stopping criterion that checks whether both the bus and pedestrian graphs are fully connected. We can check this by keeping track of the number of connected components in both union-find data structures. If both graphs have only one connected component left, we can stop iterating through the edges since union operations will no longer be possible. The reason is that once two components are connected, they will never be disconnected again, meaning that once we reach one connected component, it will stay that way (otherwise we would form a cycle).

In the end, the function has to subtract the number of kept edges from the total number of edges to get the number of removed edges. The function then returns this number as the final output.

1.4 Optimizations

The main optimization we made to the algorithm was checking whether the graphs were fully connected during iterating as opposed to checking it in the end. This both allowed us to stop iterating earlier (although not changing the time complexity in theory) and also avoid unnecessary find operations which would have $O(|V|)$ time complexity. This is done by first initiating two variables *bus_components* and *pedestrian_components* to $|V|$ (the number of vertices). In the main loop we then decrement the corresponding variable by 1 if we connect two components. With this technique we can check in constant time whether both graphs are fully connected by checking if

both variables are equal to 1. If they are not, we continue iterating through all the edges. If they still aren't in the end, we return -1 since it is impossible to connect all vertices in both graphs.

2 Correctness Analysis

Why is your algorithm correct? Provide precise arguments, supported by formal reasoning whenever needed. Think of an explanation to a critical colleague why your algorithm is correct. For examples, take a look at correctness proofs in the textbook (Algorithms Illuminated) or solutions to the weekly exercises.

3 Complexity Analysis

What is the big O complexity of your algorithm (including reading in the data and making the data structure)? The analysis should be written in terms of the given input parameters. For example, if the input includes sets V of vertices and E of edges the complexity analysis should result in a big-Oh complexity using V and E . Include an explanation of your answer.

Reading in the data

Theorem 1. *Complexity of reading in the data is $O(|E|)$ where $|E|$ is the number of edges in the graph.*

Proof. `sys.stdin.read()` reads through all the characters in the input once, which takes $O(c)$ time where c is the number of characters in the input. The `.split()` function splits the string on whitespace characters, which is also done in $O(c)$ time, since it has to check each character once. `map(int, ...)` then converts each of the k substrings into integers in $O(k)$ time. The building of tuples then iterates through the list of integers once more, making slices of 3 integers at a time (one for every edge), which takes $O(|E|)$ time. Since the file contains $3|E| + 2$ integers and c and k are both proportional to the number of integers, we can conclude that reading the data takes $O(|E|)$ time in total. $\square$

Main algorithm

Theorem 2. *Complexity of the main algorithm is $O(|E| \log |E|)$.*

Proof. **Initialization:** Initializing the union-find data structures (`parent_bus = list(range(n))` and `parent_ped = list(range(n))`) takes $O(|V|)$ time each, since creating a list of size n parses through all n elements once. Both initializations of the number of components clearly happen in constant time. $\square$

4 Reflection

Building the algorithm

I first wrote down everything I could have used from what we had discussed in the lectures or used in the weekly assignments. I came to the conclusion of using Kruskal's algorithm pretty fast but didn't immediately know what would be the best way to use it. I struggled quite long with building the right algorithm that would produce the right output. My first idea was to separate the graph into two subgraphs first and first iterate through the bus graph remembering the combined edges used and then running Kruskal's algorithm again on the pedestrian graph while marking the used edges in the previous iteration as *required edges* and adding them to the connected edges immediately. After getting the wrong output I realized that the idea wasn't going to work since the algorithm counted the same edges twice and was so computationally expensive that it had trouble computing the output for only one file. I scratched the idea and thought about how to avoid running Kruskal's algorithm twice so the idea of simultaneous algorithm without prior graph separation came to mind.

Speeding up the algorithm

The last adjustment to the algorithm was speeding it up by computing the number of components which was useful to check whether both busses and pedestrians were able to reach all stages while iteration. I had previously computed it in the end by checking the number of edges in the constructed bus and pedestrian graph. This was the last optimization I could think of for my approach but I do not reject the idea of existence of a better algorithm for this problem. For example, there surely exists a library for python or another language that supports the (simultaneous) Kruskal's algorithm better.